Symbol Store Architecture

1 Declaring symbolic expressions

1.1 shared API

To be adjusted/extended depending on the use cases and the implementation needs.

```
class Expression {
public:
   class Symbol {
   public:
      Symbol(std::string const& name);
      std::string const& getName() const;
   }

   using ReturnType = std::variant<Expression, Symbol, bool, int, float, std::string>;
   using ArgumentType = ReturnType;

   Expression(std::string const& head, std::vector<ArgumentType> const& arguments);

   std::vector<ArgumentType> const& getArguments() const;
   std::string const& getHead() const;
}
```

:

```
class Engine {
public:
  ReturnType evaluate(Expression::ArgumentType const& arg);
  ReturnType evaluate(Expression const& e); // for convenience, to be able to write
                                            // evaluate({"Function", ...})
}
```

1.2 Expression Builder

1.2.1 Convenience Functions

Here is an example of an expression construction and evaluation as we support it in the base API:

```
std::get<Type>(evaluate({"function", {Expression{"function1", {arg1, arg2}},
    Expression{"function2", {Expression::Symbol{"A"}}}}}));
```

Even though the implementation is in C++, the expressions are also intended to be exposed to diverse host languages such as Python or Lisp. For a language like Lisp, there is a straightforward mapping between the expression grammar we expose and the host language syntax. But for other languages, including C++ or Python, building directly expressions might affect the readibility. We should consider diverse ways to wrap the expression construction into a syntax which is closer to the usual language syntax of the host language. Here below are some of the proposals for it.

Not every host language and not every use case (domain) would benefit from either A, B or C. This is something to discuss if any of them would make sense to be implemented. Moreover, they don't necessarily have to be implemented for every engine backend.

A. Static functions

```
function<Type>(function1(arg1, arg2), function2(symbol("A")));
//we can also generalise to:
func<Type>("function")(func("function1")(arg1, arg2), func("function1")(symbol("A")));
```

It could be implemented by generating static functions in native c++ and additional declration to other languages (swig) from C-processor macro calls when defining the built-in functions.

B. Object-Oriented syntax

```
arg1.function1(arg2).function<Type>(symbol("A").function2());
```

This one could be implemented in a similar way with the static functions, but with some additional needs.
To be able to add them as methods of the expression class, the expression class need to derivate from a templated BuiltInFunction class which both define the symbolic built-in functions to declare as the same time as defining methods to expose them.

C. Operator Overloading

to be discussed
It would allow to simplify the syntax in a way to make it more readable while keeping it very close to the S-expression grammar from our base API.

1.2.2 JSON Serialization

In addition to syntax sugar implemented in the various host languages, we can also implement serialization from and to a JSON format similar to the ExpressionJSON used by the Wolfram system (https://reference.wolfram.com/language/ref/format/ExpressionJSON.html). It would allow us to share a common syntax which would be directly available in any host language (just exposing the loader function). Not only that but it would also be a straightforward representation to write/read into files for the tests framework and for storing expressions into the database engine.

In longer term, we should consider, as a possible alternative, the serialization of a custom binary format closer to our native expression implementation. it could be more efficient to load and store if this is a bottleneck.

2 Storing symbolic expressions

2.1 Symbolic Database

Expressions are stored as tuples in a relational database engine.
We implement the query operators and table management functions as symbolic functions.
We need to support basic operations for updating tables and tuples in the database:

`InsertInto, Delete`

Additionally we need to support operators to create, modify and access a table and table properties:

`CreateTable, DeleteTable, AddColumn, RemoveColumn, NumColumns, ColumnIndex, ColumnName`

We manipulate a table as an opaque symbol and any result from a transaction as an expression. They can both be pass as argument to the various query operators.
We can then write any of the examples below as valid expressions:

```
Expression{"CreateTable", {Expression::Symbol{"T1"}}};
Expression{"AddColumn", {Expression::Symbol{"T1"}, "ColA", "Number"}};
Expression{"Insert", {Expression::Symbol{"T1"}, {1, "name", Expression{"+", 1, 2}}}};
Expression{"Stringify", {{"Project", {Expression::Symbol{"T1"}, {"ColA"}}}}};
```

2.2 Serialization

Beside creating the database from code, we should additionally support writing to and loading from disk:

- write/load binary files from the tuple raw data stored in database (specific to each backend)

- load schema from file (json etc). can be common code to every backend as far as we have the same API for creating tables/columns
- load tuples from file (tbl, csv etc). can be common code to every backend as far as we have the same API for inserting tuples

2.3 Implementation

If it is possible to share the storage manager layer with every backend and expose only functions to extract data for the query processor, it will allows to keep the whole storage data structure / storage operators / serialization implemented only once.
To be confirmed if this approach is possible, it depends on the needs for every backend implementation.
If confirmed, need to define the interface for the storage manager

3 Evaluating symbolic expressions

We are not evaluating only single expressions. We want to support **bulk evaluation**. We should be able to evaluate efficiently all the expressions in a column for a table.
There are also additional constraints for the implementation that we should consider.

3.1 The user need some form of control on when to do the evaluation

An example use case is if the user wants to insert an interpolation as a missing data. This expression should not be evaluated at the time of the insert, instead it should get the neighbour values to interpolate only when the value is needed.

- optional lazy evaluation: we don't necessarily evaluate at insert (for example using an alternative insert function)
- in that case, it will not be evaluated until the field value is needed by any operation
- additionaly, the user can explicitely request to evaluate a whole table or single column

3.2 The user might require to keep track of unevaluated expressions

One example use case is if the user is waiting for a tuple or a column to be fully evaluated before extracting it and passing it to the next process.

- we need a special query operator or criteria to find tuples containing yet un-evaluated expressions

3.3 The user need some form of control on what to do with the result of the evaluation

The default behaviour when asking to evaluate expressions on a table is to return temporary evaluated tuples and leave untouched the stored expressions. Evaluations can be made definitive in the table by calling additional delete/insert operations.
One alternative approach would be to modify directly the field stored in the database (permanent evaluation) but it would give less flexibility to the user.

4 Built-in functions

4.1 Functions implemented for each backend

The following categories of functions are specific to the underline implementation of the backend query processor. They will have to be implemented separately for each backend.

4.1.1 Query operators

```
Select(:table, column, predicate)
Project(:table, columns[])
Aggregate(:table, column, operator)
Sort(:table, column, comparator)
Count(:table, column)
```

4.1.2 Table management

```
CreateTable(:table)
AddColumn(:table, columnName, dataType)
RemoveColumn(:table, columnName)
```

4.1.3 Table information

```
NumColumns(:table)
ColumnIndex(:table, columnName)
ColumnName(:table, columnIndex)
```

4.1.4 Table manipulation

```
Insert(:table, tuple[])
InsertUnevaluated(:table, tuple[])
Delete(:table, column, predicate)
EvaluateColumn(:table, column)
```

4.2 Functions independent from the backend implementation

The following categories of functions are not directly related with the query processor backend implementation. However, they would likely have to be implemented for each backend (to leave more control to the backend implementation) but it could alternatively be implemented as common code if there is a common API to define built-in native functions.

4.2.1 Arithmetic, comparison, logic

```
+, -, *, /    [or Plus, Substract, Times, Divide?]
<, >, <=, >=, ==, !=  [or Greater, Less, GreaterEqual, Greater, Equal, NotEqual?]
And, Or, Not
```

4.2.2 Collections

```
List(arg0, ..., argN)     [or Vector? or Tuple?]
Extract(list, index)
Map(function, list0, ..., listN)
```

4.2.3 Symbolic operators

```
Stringify(expression)
IsEvaluated(arg)
Lambda(arguments[], body)
```

4.2.4 Interpolation

Linear interpolation of value Y at position X (axis X being typically the time series) based on neighbour points prev(X,Y) and next(X,Y)

```
Interpolate(prevX, prevY, nextX, nextY, targetX)
Interpolate(columnX, columnY, valueX)
```

It will be possible to express it purely as a compound expression from other built-in functions

4.3 Function namespace

We have several open questions to discuss regarding the function namespace.

- Which ones of those functions should to be supported on every backend, which ones are specific to some backends?
- Do we separate the namespace for functions and variables (lisp-2) or merge them into a single symbol namespace (lisp-1)?
- Do we support (and how) function overloading?